Excelente, Douglas — vou estruturar isso como um **plano de ação realista de adoção de automação**, com nível de detalhe suficiente para você executar manualmente passo a passo e também alimentar um agente de IA.

☑ 1. PLANO DETALHADO (PASSO A PASSO EXECUTÁVEL)

◇ FASE 0 — Fundamentos da Estratégia (ANTES DE CONFIGURAR)

> Objetivo: evitar automação sem valor

Tarefas

1. Definir **escopo inicial de automação**
    - Escolher **1 fluxo crítico** (ex: login, cadastro, compra)
    - Critérios:
        - Alta frequência de execução
        - Alto valor de negócio
        - Baixa instabilidade
2. Definir **objetivo da automação**
    - Reduzir tempo de regressão?
    - Aumentar cobertura?
    - Detectar bugs cedo?
3. Criar **primeiro backlog de testes**
    - Escrever manualmente:
        - 5 a 10 cenários críticos
    - Exemplo:

◇ FASE 1 — Preparação do Ambiente (DO ZERO)

1. Criar conta no GitHub

- Acesse: https://github.com
- Definir:
    - Username profissional
    - Email

---

2. Instalar Git (Ubuntu)

```
sudo apt update
sudo apt install git -y
```

Verificar:

```
git --version
```

Configurar:

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
```

---

3. Instalar Python 3.11+

```
sudo apt install python3.11 python3.11-venv python3.11-dev -y
```

Verificar:

```
python3.11 --version
```

---

4. Instalar VSCode

```
sudo snap install code --classic
```

Instalar extensões:

- Python
- Pylance
- GitLens

◇ FASE 2 — Criação do Projeto

1. Criar diretório

```
mkdir automacao-testes
cd automacao-testes
```

---

2. Criar ambiente virtual

```
python3.11 -m venv venv
source venv/bin/activate
```

3. Criar estrutura inicial (IMPORTANTE)

```
mkdir -p tests/pages
mkdir -p tests/transactions
mkdir -p tests/specs
mkdir -p tests/fixtures
touch pytest.ini
touch requirements.txt
```

Estrutura final:

```
automacao-testes/
│
├── tests/
│   ├── pages/
│   ├── transactions/
│   ├── specs/
│   ├── fixtures/
│
├── pytest.ini
├── requirements.txt
└── README.md
```

4. Configurar dependências

Editar requirements.txt:

```
selenium
pytest
webdriver-manager
python-dotenv
```

Instalar:

```
pip install -r requirements.txt
```

5. Instalar Guará Framework

```
pip install guara
```

> Caso o pacote tenha repositório específico, adaptar depois.

◇ FASE 3 — Integração com GitHub

1. Criar repositório no GitHub

- Nome: automacao-testes
- Marcar:
  - ✅ README
  - ✅ .gitignore (Python)

---

2. Conectar local ao remoto

```
git init
git add .
git commit -m "estrutura inicial automacao"
git branch -M main
git remote add origin https://github.com/SEU_USUARIO/automacao-testes.git
git push -u origin main
```

◇ FASE 4 — Configuração Base de Testes

1. Criar fixture de navegador

Arquivo: tests/fixtures/driver.py

```
from selenium import webdriver
from webdriver_manager.chrome import ChromeDriverManager

import pytest

@pytest.fixture
def driver():
    driver = webdriver.Chrome(ChromeDriverManager().install())
    driver.maximize_window()
     yield driver
     driver.quit()
```

---

2. Criar primeiro Page Object

Arquivo: tests/pages/login_page.py

```
from selenium.webdriver.common.by import By
```

```
2
3  class LoginPage:
4
5      def __init__(self, driver):
6          self.driver = driver
7
8      def acessar(self):
9          self.driver.get("https://sua-url.com/login")
10
11      def preencher_usuario(self, usuario):
12          self.driver.find_element(By.ID,
"username").send_keys(usuario)
13
14      def preencher_senha(self, senha):
15          self.driver.find_element(By.ID,
"password").send_keys(senha)
16
17      def clicar_login(self):
18          self.driver.find_element(By.ID, "login").click()
```

3. Criar primeira Transaction

Arquivo: tests/transactions/login_transaction.py

```
1  from tests.pages.login_page import LoginPage
2
3  class LoginTransaction:
4
5      def __init__(self, driver):
6          self.page = LoginPage(driver)
7
8      def realizar_login(self, usuario, senha):
9          self.page.acessar()
10          self.page.preencher_usuario(usuario)
11          self.page.preencher_senha(senha)
12          self.page.clicar_login()
```

4. Criar primeiro teste

Arquivo: tests/specs/test_login.py

```
1  def test_login_valido(driver):
```

```
    from tests.transactions.login_transaction import
LoginTransaction

    login = LoginTransaction(driver)
    login.realizar_login("usuario", "senha")

    assert "dashboard" in driver.current_url
```

5. Executar teste

```
pytest -v
```

◇ FASE 5 — Evolução com Valor (MUITO IMPORTANTE)

Tarefas estratégicas

1. Criar **padrão de naming**
    - test_login.py
    - login_page.py
    - login_transaction.py
2. Separar:
    - Page → elementos
    - Transaction → fluxo
    - Test → validação
3. Criar testes prioritários:
    - Smoke tests
    - Fluxos críticos
    - Regressão básica

☑ 2. SUMÁRIO (COMO O PLANO FOI CONSTRUÍDO)

Esse plano segue uma abordagem de arquitetura de QA moderna:

◇ Princípios usados

- **Começar pequeno (MVP de automação)** → evita fracasso
- **Foco em valor de negócio** → não automatizar tudo
- **Separação de responsabilidades**
    - Page Object → UI
    - Transaction → fluxo

    - Test → validação
- **Evolução incremental**

◇ Fluxo lógico do plano

1. Definir estratégia → evitar desperdício
2. Preparar ambiente → base técnica
3. Criar estrutura → escalabilidade
4. Fazer primeiro teste → validação prática
5. Evoluir gradualmente → maturidade

☑ 3. BOAS PRÁTICAS (NÍVEL SÊNIOR)

◇ Estratégia de Automação

✅ Não automatize:

- Testes instáveis
- UI muito volátil
- Cenários raros

✅ Priorize:

- Testes repetitivos
- Fluxos críticos
- Regressão

◇ Arquitetura

✓ Nunca misturar:

- Page com validação
- Test com lógica complexa

✓ Use:

- Page Object (UI)
- Transaction (fluxo)
- Fixture (setup)

◇ Qualidade dos testes

✓ Sempre ter:

- Assert claro
- Nome descritivo
- Independência

❌ Evitar:

```
assert True
```

◇ Manutenção

✓ Use seletores estáveis:

- id > name > css > xpath

✓ Evite:

- XPath longo
- Dependência de texto

◇ Evolução futura (próximos passos)

Depois desse setup:

1. ✅ Adicionar logs
2. ✅ Screenshots em falha
3. ✅ Execução paralela (pytest-xdist)
4. ✅ CI/CD (GitHub Actions)
5. ✅ Relatórios (Allure)

🚀 CONCLUSÃO

Você não está apenas criando automação — está criando uma **plataforma escalável de QA**.

Esse plano garante:

- Início rápido
- Baixo risco
- Evolução contínua

- Geração de valor real

✅ Se quiser, posso te entregar o próximo passo já pronto:

👉 Pipeline CI/CD no GitHub Actions
👉 Estrutura completa com Allure + logs
👉 10 testes reais prontos para copiar

Só me diga: **qual sistema você pretende automatizar primeiro (web, API, interno, público)?**